An Analysis of Class Imbalance Challenges in Predictive Mutation Testing

Sara Balderas-Díaz(✉), Gabriel Guerrero-Contreras, Pedro Delgado-Pérez, and Inmaculada Medina-Bulo

Department of Computer Science and Engineering, University of Cadiz, Av. Universidad de Cádiz, 10, Puerto Real, 11519 Cádiz, Spain
{sara.balderas,gabriel.guerrero,pedro.delgado,inmaculada.medina}@uca.es

Abstract. This study evaluates the efficacy of Predictive Mutation Testing (PMT) models, focusing on the impact of preprocessing, class rebalancing, and model selection on predictive accuracy. By filtering unreached mutants, artificially inflated metrics caused by trivial correlations were minimized, enhancing the ability of the model to identify complex patterns. While ensemble models like CatBoost and XGBoost showed high effectiveness in detecting killed mutants, their performance in identifying survived mutants was limited, underscoring the need for refined strategies to address class imbalance. The study further identifies that previous PMT research often lacks class-specific metrics, such as the Matthews Correlation Coefficient, essential for assessing balanced performance. This oversight can yield misleading evaluations by favoring the majority class, which may falsely indicate high model reliability. Although SMOTE rebalancing improved recall for survived mutants, balancing sensitivity and precision remains challenging. The findings advocate for the development of more balanced datasets in future PMT research to enhance both accuracy and robustness in real-world testing scenarios.

Keywords: Predictive Mutation Testing · class imbalance · machine learning · ensemble models · software testing

1 Introduction

Software testing is a critical phase in the software development lifecycle, aimed at identifying and eliminating faults to ensure high-quality software products. Mutation testing has emerged as a powerful technique to evaluate the effectiveness of test suites by introducing small modifications, or mutations, to the code of the program and determining if these changes are detected by the existing tests [4]. However, despite its effectiveness, mutation testing is computationally intensive, as it requires running the entire test suite against a large number of mutated versions of the software. Predictive Mutation Testing (PMT) leverages machine learning models to predict the outcome of mutations(whether a mutant

N. T. Nguyen et al. (Eds.): ACIIDS 2025, LNAI 15684, pp. 319–331, 2025.
https://doi.org/10.1007/978-981-96-6005-6_23

is killed or survived) thereby reducing the need to execute the full test suite and significantly lowering computational demands [14].

Recent advancements show that ML models trained on features such as code structure and test coverage can accurately predict killed mutants, validating PMT as a viable alternative to traditional mutation testing [7,9,14]. However, the effectiveness of these models is influenced by factors like data preprocessing, class imbalance, and model selection, with a pronounced imbalance between killed and survived mutants potentially skewing datasets and biasing models toward the majority class [1].

An often-overlooked aspect in PMT research is the use of detailed, class-specific performance metrics, which are essential for assessing model efficacy in binary classification tasks. While aggregate metrics such as overall accuracy or F1 score provide a broad measure of model performance, they can obscure important class-specific behaviors [10]. This issue is particularly problematic in PMT, where both killed and survived mutants play distinct roles in assessing the quality of a test suite. If a model primarily excels at identifying killed mutants but performs poorly with survived mutants, it may lead to an incomplete and potentially misleading evaluation of the testing needs of the software.

In this study, an in-depth analysis is performed on one of the primary PMT datasets [14] to assess the validity of models that can be constructed from these data, with particular attention to class-specific performance. Unreached mutants (those not covered by any test cases and therefore trivially categorized as survived) are filtered out to reduce potential biases that may result in artificially inflated performance metrics. To address the class imbalance between killed and survived mutants and improve accurate detection of both classes, various rebalancing methods, including RandomUnderSampler (RUS), Synthetic Minority Over-sampling Technique (SMOTE) and Edited Nearest Neighbors (ENN), are explored.

The rest of this paper is organized as follows. In Sect. 2, the literature on PMT is reviewed. The dataset, preprocessing steps, rebalancing techniques, and the machine learning models used in this study are described in Sect. 3. In Sect. 4, the results are presented and analyzed. Finally, conclusions and future work are provided in Sect. 5.

2 Related Work

Foundational work in PMT [14], based on the Propagation-Infection-Execution (PIE) Theory, established a framework that supports feature selection in both cross-project and cross-version contexts. This approach demonstrated significant predictive capability, achieving an area under the curve (AUC) exceeding 0.80 across 163 real-world projects. Such results underscored the efficacy of PMT in maintaining high accuracy with limited feature adjustments, emphasizing the critical role of execution, infection, and propagation metrics in driving model performance.

Subsequent research has introduced cross-project PMT models that significantly reduce the computational load of traditional mutation testing [9]. Evaluated on 654 Java projects, one such model achieved an average time reduction of 28.7 times. By integrating a broad range of static and dynamic features and employing random forest classifiers, the model was especially effective at predicting killed mutants. The study also underscored the importance of dynamic features (e.g., count of test methods covering a mutation) in enhancing predictive precision.

The adaptability of PMT was confirmed in its application to the C programming language [5], where the study achieved an AUC above 0.90 and a prediction error below 1%. These results underscore the versatility of PMT across diverse programming environments and suggest its broader applicability beyond Java.

Ensemble learning techniques have emerged as an effective solution to address class imbalance [1]. By incorporating synthetic data rebalancing through ADASYN and ensemble classifiers like Random Forest and Gradient Boosting, this model achieved an AUC of 0.613, lowering the initial expectations. This approach showed that focusing on reached mutants, together with rebalancing methods, can significantly improve the reliability of PMT predictions, as unreached mutants (those not covered by any test cases) often distort performance metrics and reduce predictive reliability.

The integration of natural language features enhances the adaptability of PMT. The Seshat approach leverages source code names and comments with natural language processing to predict kill matrices more granularly [8]. Using deep learning (DL) models with gated recurrent units (GRUs) and word embeddings, Seshat reduces mutation analysis costs substantially, achieving up to a 68-fold reduction while sustaining an F1 score of 0.86 when applied to machine-generated test suites.

Transformer-based approaches, such as MutationBERT [7], have further refined PMT by encoding mutations as token-level differences within the context of both source and test code. MutationBERT improved precision by 30% and reduced verification time by 33%, particularly in cross-project applications.

Meta-learning and hybrid models improve PMT by mitigating data imbalance and enhancing predictive reliability. In [3], a meta-learning approach using Logistic Regression as a meta-classifier outperformed traditional models like XGBoost, CatBoost, and Random Forest, with statistical significance validated by Wilcoxon and Friedman-Nemenyi tests.

Despite considerable advancements in PMT, the focus in much of the existing literature remains on overall model accuracy, often overlooking the critical need for class-specific performance assessments. This limitation is particularly evident in models trained on imbalanced datasets, where the underrepresentation of reachable survived mutants can lead to biases in the models.

3 Material and Methods

3.1 Dataset Overview

The dataset consists of `.arff` files representing mutants from 154 Java projects, totaling 519,530 mutants [14], which were transformed into `.csv` format. Each mutant is labeled as either survived or killed. This dataset provides a robust foundation for PMT. Notably, project sizes vary significantly: `evernotesdk-java` has 62,627 mutants, while `byteunits` includes just 31, with an average of 3373.57 mutants per project. Of the total mutants, 105,069 (20.22%) are killed, and 414,461 (79.78%) are survived, reflecting a strong class imbalance.

Mutant features include test coverage, statement complexity, structural characteristics, and oracle-related metrics, all critical for assessing differences between killed and survived mutants. Specifically, features such as `numExecuteCovered` and `numTestCovered` track the execution frequency and test suite coverage for each mutant, while `typeOperator` indicates the mutation operator type (e.g., logical connector). In addition, information about structural complexity, such as `infoComplexity`, and package dependencies, `Ca` (afferent couplings) and `Ce` (efferent couplings). Other notable features include `depInheritance`, `depNestblock`, and oracle-related metrics such as `numMutantAssertion` and `numClassAssertion`.

3.2 Data Preprocessing

The dataset initially contained 383,583 *unreached mutants*, defined by a zero count in the `numCovered` variable (i.e., `numCovered == 0`). Including these mutants could bias the model, as their detection would be trivial based on the `numCovered` value alone, circumventing the need for the model to learn complex relationships. To avoid artificially inflated performance results, these mutants were excluded. This decision aligns with previous studies, which suggest that unreached mutants can skew PMT outcomes [1].

Following this filtering step, the dataset size reduced from 519,530 instances to 135,947, consisting of 30,878 mutants in the survived class and 105,069 in the killed class (Table 1). The filtered dataset presents a class imbalance, with 77.29% of instances labeled as killed and 22.71% as survived. This imbalance could significantly influence model performance.

Table 1. Distribution of killed and survived instances in the full and filtered datasets.

Dataset	Killed N	% Killed	Survived N	% Survived	Total N
Full Dataset	105,069	20.22%	414,461	79.78%	519,530
Filtered Dataset	105,069	77.29%	30,878	22.71%	135,947

Data partitioning is conducted separately for each project, applying a random split to a collection of 154 Java projects. This method ensures that instances from

the same project do not appear in both the training and testing sets, which would otherwise inflate model performance due to data leakage. With this approach, 80% of the projects (124) are allocated to the training set, while the remaining 20% (30) are designated for testing. Table 2 displays the distribution of killed and survived mutants in both sets. The training set includes 91,273 killed mutants (77.10%) and 27,111 survived mutants (22.90%), while the test set contains 13,796 killed mutants (78.55%) and 3,767 survived mutants (21.44%). It should be noted that the killed-to-survived mutant ratio in each set closely aligns with the filtered dataset, preserving the natural class imbalance.

Table 2. Distribution of mutants (killed and survived) in the dataset after random partitioning for training and testing.

Dataset	Killed N	Killed %	Survived N	Survived %	Total N
Training	91,273	77.10%	27,111	22.90%	118,384
Test	13,796	78.55%	3,767	21.44%	17,563

Further analysis of features revealed deviations from normality and the presence of outliers. Violin plots for key features such as `LOC`, `methodReturn`, `numTestCover`, and `mutantAssert` (Fig. 1) indicated irregular distributions. These deviations normality assumptions and suggest that outliers may influence analysis and model performance. To address this, the features were normalized using the `RobustScaler`, which is resilient to outliers, ensuring robustness in the data preparation process. Data scaling was performed based on the range of the training dataset alone. In this way, the model remains strictly blind to the test set characteristics during training, allowing for an unbiased evaluation on unseen data.

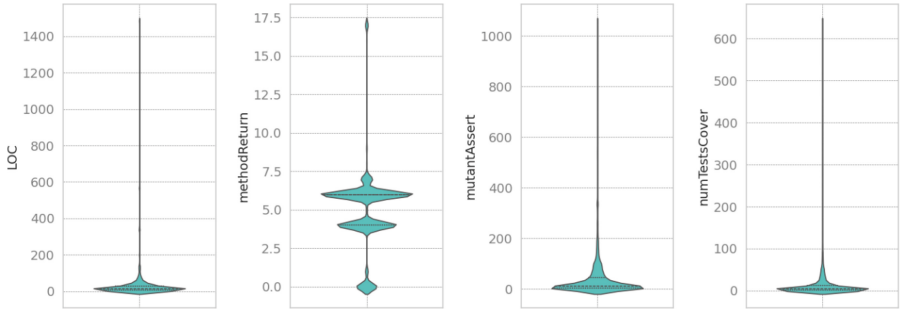

Fig. 1. Violin plots illustrating the distribution of key features of the dataset.

3.3 Class Imbalance

To address the significant imbalance between killed and survived mutants and prevent biased model predictions favoring the majority class, three rebalancing techniques were applied exclusively to the training set: Synthetic Minority Over-sampling Technique (SMOTE), RandomUnderSampler (RUS), and Edited Nearest Neighbors (ENN). SMOTE generated synthetic samples for the minority class (survived mutants) by interpolating between existing instances, achieving a balanced 50% representation for each class [2]. RUS balanced the dataset by randomly removing instances from the majority class (killed mutants), also producing a 50% distribution [12]. In contrast, ENN selectively removed mislabeled or ambiguous samples from the majority class, enhancing class separation and refining the decision boundary. Unlike SMOTE and RUS, ENN achieved a less aggressive balance, with mutants killed comprising 71. 23% and mutants survived that represented 28. 77% of the dataset [15]. Table 3 summarizes the distribution of mutants killed and survived after applying these rebalancing techniques.

Table 3. Comparison of class instances and rebalancing impact in training data.

Dataset	Killed N	Killed %	Survived N	Survived %	Total N
SMOTE	91,273	50%	91,273	50%	182,546
RandomUnderSampler	27,111	50%	27,111	50%	54,222
ENN	67,128	71.23%	27,111	28.77%	94,239

3.4 Evaluation Metrics

To assess the performance of each model, Precision, Recall, F1 Score, Matthews Correlation Coefficient (MCC), and Area Under the Receiver Operating Characteristic Curve (AUCROC) are used and defined as follows [11].

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{F1 Score} = 2\,\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \qquad \text{MCC} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}}$$

3.5 Machine Learning Models

A Decision Tree (DT) model is used as a baseline for its simplicity and interpretability, serving as a reference to evaluate the effectiveness of more complex models and rebalancing methods. Subsequently, additional machine learning algorithms are employed, including k-Nearest Neighbors (kNN), Logistic Regression (LR), Random Forest (RF), Extreme Gradient Boosting (XGBoost), LightGBM, AdaBoost, Gradient Boosting, Extra Trees, Ridge Classifier, CatBoost, and Multi-layer Perceptron (MLP) [6,13].

4 Results and Discussion

4.1 Impact of Unreached Mutant Filtering

The performance impact of filtering out unreached mutants is clearly reflected in the baseline model metrics. When applied to the full dataset, which includes unreached mutants (Table 4), the model achieves a slightly higher F1 score (0.7661) compared to the filtered dataset (0.7632). More notably, MCC and AUCROC metrics diverge significantly, the full dataset yields an MCC of 0.7386 and an AUCROC of 0.8907, while the filtered dataset reaches only 0.0628 and 0.5353, respectively, values close to random guessing. This disparity suggests that unreached mutants simplify the task of the model by creating a predictable correlation between unreached status (`numCovered == 0`) and the label, resulting in artificially inflated performance. In contrast, removing these mutants forces the model to capture more intricate data patterns, leading to a substantial drop in AUCROC and MCC.

Table 4. Comparison of performance metrics for the baseline model on the full and filtered datasets.

Model	F1 Score	Precision	Recall	MCC	AUCROC
Full dataset	0.7661	0.8005	0.7345	0.7386	0.8907
Filtered dataset	0.7632	0.8019	0.7281	0.0628	0.5353

Further, individual class metrics (Table 5) underscore the effect of filtering. In the full dataset, the survived class metrics reach approximately 0.97–0.98, indicating reliable classification within this category. However, in the filtered dataset, these metrics drop sharply, with precision, recall, and F1 falling to 0.26, 0.34, and 0.29, respectively, highlighting the increased difficulty in detecting survived mutants when all mutants are reachable, as the unreached indicator no longer provides a straightforward means for classification. For the killed class, performance remains relatively stable across both datasets, with precision around 0.80 and recall near 0.73. These findings align with prior studies, which suggest that including unreached mutants can lead to an inflated assessment of model performance. By excluding these trivial instances, model evaluation offers a more realistic reflection of the model ability to discern complex relationships, avoiding over-reliance on biases within the dataset.

4.2 Evaluation of Rebalancing Methods

Table 6 compares the performance of the baseline model using different balancing methods (SMOTE, RUS, and ENN) on the filtered dataset. SMOTE achieved the highest F1 Score (0.7369) and Recall (0.6730), effectively balancing sensitivity to identifying killed mutants and minimizing false negatives. Although ENN demonstrated a marginally higher Precision (0.8146 vs. 0.8143 for SMOTE), SMOTE

Table 5. Performance metrics by class for the baseline model on full and filtered datasets.

Dataset	Precision		Recall		F1 Score	
	Survived	Killed	Survived	Killed	Survived	Killed
Full dataset	0.97	0.80	0.98	0.73	0.97	0.77
Filtered dataset	0.26	0.80	0.34	0.73	0.29	0.76

demonstrates superior Recall and F1 Score, providing a more balanced performance for applications that prioritize the detection of killed mutants. ENN, while competitive, presented a lower Recall (0.6258) and F1 Score (0.7078) relative to SMOTE. Its slightly higher AUCROC (0.5624) did not offset the reduced sensitivity. RUS performed the worst, with the lowest F1 Score (0.6636) and Recall (0.5642), likely due to its aggressive undersampling. The MCC values across all methods are relatively low, which may indicate challenges in balancing the predictions effectively for this specific task, despite improvements introduced by the rebalancing techniques. Therefore, based on the analysis, SMOTE is selected as the rebalancing method because consistently outperforms the other methods, demonstrating a more balanced and reliable performance for the given dataset.

Table 6. Evaluation of model performance metrics using different rebalancing techniques on the filtered dataset.

Method	F1 Score	Precision	Recall	MCC	AUCROC
SMOTE	0.7369	0.8143	0.6730	0.0955	0.5596
RUS	0.6636	0.8054	0.5642	0.0535	0.5362
ENN	0.7078	0.8146	0.6258	0.0874	0.5624

4.3 Hyperparameter Tuning

The best hyperparameters for each model are obtained through a grid search using a 10-fold cross-validation strategy, which identified optimal settings to improve performance in mutation testing. Table 7 summarizes these results.

4.4 Comparative Assessment of Model Performance

Table 8 provides a broad assessment of the effectiveness of each model across key metrics. The ensemble-based models, including CatBoost, XGBoost, and RandomForest, display the most balanced performance overall, achieving high F1 scores (0.8337 to 0.8439) and maintaining a favorable precision-recall balance. CatBoost stands out with an F1 score of 0.8439, indicating robust predictive

Table 7. Best hyperparameter configurations for each model.

Model	Hyperparameters
DecisionTree	`max_depth`=10, `min_samples_leaf`=1, `min_samples_split`=2
kNN	`metric`=manhattan, `n_neighbors`=7, `weights`=distance
LogisticRegression	`C`=0.01, `penalty`=l1
RandomForest	`max_depth`=20, `min_samples_split`=2, `n_estimators`=200
XGBoost	`learning_rate`=0.2, `max_depth`=10, `n_estimators`=200
LightGBM	`learning_rate`=0.1, `max_depth`=20, `n_estimators`=200
AdaBoost	`learning_rate`=1.0, `n_estimators`=100
GradientBoosting	`learning_rate`=0.1, `max_depth`=10, `n_estimators`=200
ExtraTrees	`max_depth`=None, `min_samples_split`=5, `n_estimators`=100
RidgeClassifier	`alpha`=10
CatBoost	`depth`=10, `iterations`=200, `learning_rate`=0.1
MLP	`activation`=tanh, `hidden_layer_sizes`=(50,) `learning_rate_init`=0.001

capacity in mutation testing. However, low MCC values across the ensemble models highlight challenges in maintaining balanced performance across classes. For example, the MCC of XGBoost reaches only 0.1629, and that of CatBoost is 0.1479. These low values suggest that despite high F1 scores, the models struggle with consistent correlation between predicted and actual classes.

Table 8. Performance metrics across models for overall comparison.

Model	F1 Score	Precision	Recall	MCC	AUCROC
DecisionTree	0.7656	0.8272	0.7125	0.1470	0.6229
kNN	0.7331	0.7899	0.6839	0.0157	0.5166
LogisticRegression	0.6207	0.8018	0.5064	0.0393	0.5365
RandomForest	0.8337	0.8066	0.8628	0.1178	0.6243
XGBoost	0.8354	0.8166	0.8551	0.1629	0.6631
LightGBM	0.8250	0.8206	0.8294	0.1680	0.6686
AdaBoost	0.7533	0.8449	0.6796	0.1896	0.6544
GradientBoosting	0.8346	0.8084	0.8626	0.1267	0.6573
ExtraTrees	0.8175	0.8061	0.8292	0.1028	0.6025
RidgeClassifier	0.6317	0.8036	0.5204	0.0448	–
CatBoost	0.8439	0.8108	0.8798	0.1479	0.6585
MLP	0.7377	0.8143	0.6743	0.0958	0.5662

AUCROC scores add further insights into model distinctions, though values remain modest overall. XGBoost and LightGBM achieve the highest AUCROC scores, at 0.6631 and 0.6686, respectively, reflecting moderate discrimination ability across decision thresholds. In contrast, simpler models such as Logistic Regression and kNN exhibit even lower AUCROC scores of 0.5365 and 0.5166, respectively, underscoring their limited discrimination capability, close to random guessing. This underperformance is likely due to their inability to capture the intricate relationships and feature interactions inherent in the dataset. Logistic Regression, for instance, assumes linear separability, which limits its effectiveness when the data exhibits complex, non-linear patterns. Similarly, kNN relies on distance metrics to classify instances, making it sensitive to high-dimensional data and prone to noise.

Single-decision models, notably DecisionTree, show lower overall performance metrics, achieving an F1 score of 0.7656 and an AUCROC of 0.6229 due to reliance on individual splits, leading to overfitting or underfitting with imbalanced data. Similarly, the ridge-based classifier underperforms against ensemble models due to its linearity assumption and lacks reliable AUCROC, indicating challenges in binary class separation for mutation testing.

Table 9. Detailed class-specific performance metrics for each model.

Model	Precision		Recall		F1 Score	
	Survived	Killed	Survived	Killed	Survived	Killed
DecisionTree	0.30	0.83	0.46	0.71	0.36	0.77
kNN	0.22	0.79	0.33	0.68	0.27	0.73
LogisticRegression	0.23	0.80	0.54	0.51	0.32	0.62
RandomForest	0.33	0.81	0.24	0.86	0.28	0.83
XGBoost	0.36	0.82	0.30	0.86	0.32	0.84
LightGBM	0.35	0.82	0.34	0.83	0.34	0.83
AdaBoost	0.32	0.84	0.54	0.68	0.40	0.75
GradientBoosting	0.33	0.81	0.25	0.86	0.29	0.83
ExtraTrees	0.30	0.81	0.27	0.83	0.28	0.82
RidgeClassifier	0.23	0.80	0.53	0.52	0.32	0.63
CatBoost	0.36	0.81	0.25	0.88	0.29	0.84
MLP	0.27	0.81	0.44	0.67	0.33	0.74

Table 9 shows that all models perform better on killed mutants than survived mutants, with notable differences in precision, recall, and F1 scores between the two classes. Ensemble models, such as XGBoost and CatBoost, show modest performance on survived mutants (F1 score: 0.32 and 0.29, precision: ∼0.36), reflecting challenges in minimizing false positives and indicating potential bias against this class. LightGBM and GradientBoosting perform similarly, with F1

scores of 0.34 and 0.29. AdaBoost and Logistic Regression achieve the highest Recall (0.54) for survived mutants but still show inconsistent coverage.

In contrast, performance on killed mutants is consistently higher across all models. CatBoost and XGBoost achieve the highest F1 scores (0.84) with Precision and Recall both exceeding 0.80, effectively identifying killed mutants with minimal false positives. RandomForest, LightGBM, and GradientBoosting also perform strongly on killed mutants.

5 Conclusions and Future Work

This study provides a detailed evaluation of PMT models, focusing on the impact of dataset preprocessing, rebalancing techniques, and model selection on predictive accuracy. Filtering unreached mutants enhances pattern recognition and prevents inflated metrics from trivial correlations. Ensemble models like CatBoost and XGBoost effectively detect killed mutants, but performance drops significantly for survived mutants, highlighting the need for better strategies to handle class imbalance.

Model effectiveness in PMT is significantly limited by the reduced ability to detect survived mutants. Although previous studies report high aggregate metrics, the absence of class-specific performance measures (e.g., MCC) conceals weaknesses in identifying survived mutants, leading to biased assessments favoring the majority class and giving a misleading sense of model reliability. This issue is exacerbated by dataset imbalance, with an overrepresentation of killed and unreached mutants, limiting the model's exposure to the characteristics of survived mutants. This bias reduces practical applicability, as accurate identification of both classes is essential for comprehensive software quality evaluation. The lack of class-specific metrics has led to inadequate model assessments, concealing limitations in detecting survived mutants and providing an overly optimistic view of model effectiveness.

Class-specific analysis confirms this bias, revealing consistently lower precision and recall for survived mutants. Although SMOTE improved recall for survived mutants, maintaining overall model sensitivity and precision remains challenging. Future research should focus on constructing more balanced and diverse datasets, enabling models to generalize better across classes and enhancing the accuracy and robustness of PMT models for real-world testing environments.

Acknowledgments. This publication is part of the I+D+i grant PID2021-122215NB-C33 funded by MICIU/AEI/10.13039/501100011033 and by ERDF/EU, and by the Spanish Ministry of Science and Innovation under the Excellence Network AI4Software (Red2022-134647-T).

References

1. Aghamohammadi, A., Mirian-Hosseinabadi, S.H.: An ensemble-based predictive mutation testing approach that considers impact of unreached mutants. Softw. Testing Verification Reliab. **31** (2021). https://doi.org/10.1002/stvr.1784
2. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. J. Artif. Intell. Res. **16**, 321–357 (2002)
3. Chetna, Kaur, K.: Empirical study of meta-learning-based approach for predictive mutation testing. In: Lecture Notes in Electrical Engineering. LNEE, vol. 1078, pp. 623–635. Springer, Cham (2023). https://doi.org/10.1007/978-981-99-5974-7_50
4. Delgado-Pérez, P., Sánchez, A.B., Segura, S., Medina-Bulo, I.: Performance mutation testing. Softw. Test. Verif. Reliab. **31**(5), e1728 (2021). https://doi.org/10.1002/stvr.1728,e1728stvr.1728
5. Duque-Torres, A., Doliashvili, N., Pfahl, D., Ramler, R.: Predicting survived and killed mutants. In: Proceedings - 2020 IEEE 13th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2020, pp. 274–283. Institute of Electrical and Electronics Engineers Inc. (2020). https://doi.org/10.1109/ICSTW50294.2020.00053
6. Ganaie, M.A., Hu, M., Malik, A.K., Tanveer, M., Suganthan, P.N.: Ensemble deep learning: a review. Eng. Appl. Artif. Intell. **115**, 105151 (2022). https://doi.org/10.1016/j.engappai.2022.105151
7. Jain, K., Alon, U., Groce, A., Goues, C.L.: Contextual predictive mutation testing. In: ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 250–261. Association for Computing Machinery, Inc (2023). https://doi.org/10.1145/3611643.3616289
8. Kim, J., Jeon, J., Hong, S., Yoo, S.: Predictive mutation analysis via the natural language channel in source code. ACM Trans. Softw. Eng. Methodol. **31** (2022). https://doi.org/10.1145/3510417
9. Mao, D., Chen, L., Zhang, L.: An extensive study on cross-project predictive mutation testing. In: Proceedings - 2019 IEEE 12th International Conference on Software Testing, Verification and Validation, ICST 2019, pp. 160–171. Institute of Electrical and Electronics Engineers Inc. (2019). https://doi.org/10.1109/ICST.2019.00025
10. Owusu-Adjei, M., Ben Hayfron-Acquah, J., Frimpong, T., Abdul-Salaam, G.: Imbalanced class distribution and performance evaluation metrics: a systematic review of prediction accuracy for determining model performance in healthcare systems. PLOS Digit. Health **2**(11), e0000290 (2023). https://doi.org/10.1371/journal.pdig.0000290
11. Rainio, O., Teuho, J., Klén, R.: Evaluation metrics and statistical tests for machine learning. Sci. Rep. **14**(1), 6086 (2024)
12. Seiffert, C., Khoshgoftaar, T.M., Van Hulse, J., Napolitano, A.: Rusboost: a hybrid approach to alleviating class imbalance. IEEE Trans. Syst. Man Cybern.-Part A: Syst. Hum. **40**(1), 185–197 (2009). https://doi.org/10.1109/TSMCA.2009.2029559

13. Yang, X., Wen, W.: Ridge and lasso regression models for cross-version defect prediction. IEEE Trans. Reliab. **67**(3), 885–896 (2018). https://doi.org/10.1109/TR.2018.2847353
14. Zhang, J., Zhang, L., Harman, M., Hao, D., Jia, Y., Zhang, L.: Predictive mutation testing. IEEE Trans. Softw. Eng. **45**, 898–918 (2019). https://doi.org/10.1109/TSE.2018.2809496
15. Zhu, Y., Jia, C., Li, F., Song, J.: Inspector: a lysine succinylation predictor based on edited nearest-neighbor undersampling and adaptive synthetic oversampling. Anal. Biochem. **593**, 113592 (2020)